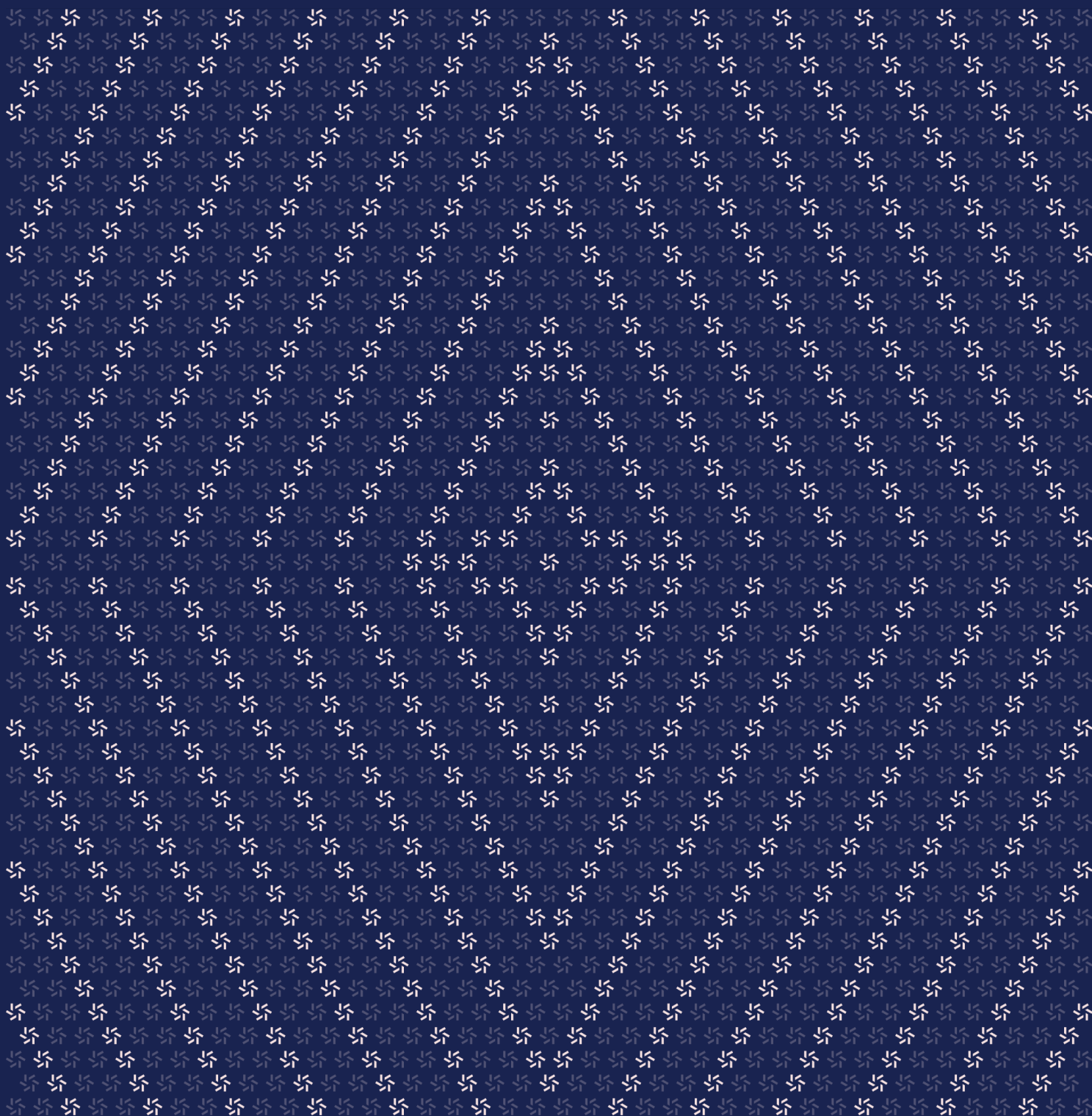


June 26, 2026

Nucleus Boring Vault

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Nucleus Boring Vault	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. The setDelegate setter does not reject the zero address	11
4. Discussion	13
4.1. Subdust orders are accepted instead of reverting	13
4.2. Fee-on-transfer tokens are not supported by the order accounting	13
4.3. Reorg risk and LayerZero block confirmation settings	14

5.	Threat Model	16
5.1.	TransitStation.sol	16
6.	Assessment Results	40
6.1.	Disclaimer	40

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Paxos Labs from June 22nd to June 23rd, 2026. During this engagement, Zellic reviewed Nucleus Boring Vault's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Do the LayerZero (LZ) components check authorization correctly?
 - Are there any DOS vectors in order execution?
 - Does the TransitStation correctly handle LZ messages?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Nucleus Boring Vault contracts, we discovered one finding, which was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Paxos Labs in the Discussion section ([4. ↗](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	1

2. Introduction

2.1. About Nucleus Boring Vault

Paxos Labs contributed the following description of Nucleus Boring Vault:

Transit is our stablecoin orchestration project. The purpose is to allow users to request a swap with a dynamic fee from one chain to another among supported routes. Signed orders approved by our backend service may be submitted and the contract will log the owed tokens on the appropriate station — bridging if necessary. Then our same service will front or otherwise make liquidity available on that destination station before executing these orders. The station never custodies inventory, instead relying on approvals from the address storing funds, and transfers directly to the recipient. It also ensures no dangling approval is left over on executions. The “owner” is assumed to be of the highest trust assumption — our multisig but Solmate RolesAuthority is used to manage other roles as necessary.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We

also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4.7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Nucleus Boring Vault Contracts

Type	Solidity
Platform	EVM-compatible
Target	nucleus-boring-vault
Repository	https://github.com/paxoslabs/nucleus-boring-vault ↗
Version	2a32b7cc3dfe83825cf9cc5a8cbe751dec95b122
Programs	transit/TransitStation.sol base/Roles/CrossChain/OAppAuth/**/*.sol helper/Pausable.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-days. The assessment was conducted by two consultants over the course of two calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✂ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
✂ Engineer
filipe@zellic.io ↗

Evan Hwang
✂ Engineer
evan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 22, 2026 Start of primary review period

June 23, 2026 End of primary review period

3. Detailed Findings

3.1. The setDelegate setter does not reject the zero address

Target	OAppAuthCore		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The setDelegate function updates the LayerZero delegate — the account authorized to configure this OApp on the endpoint — by forwarding the new address straight to the endpoint without validating it:

```
function setDelegate(address _delegate) public requiresAuth {
    endpoint.setDelegate(_delegate);
}
```

The constructor of the same contract requires the delegate to be nonzero (reverting with InvalidDelegate), but the setter does not preserve that property, allowing the delegate to be cleared to the zero address.

This matters because the contract exposes no other path to the endpoint: setDelegate is the only endpoint call it makes, and there is no passthrough for setConfig, send and receive library selection, or message recovery. The delegate is therefore the exclusive channel through which this OApp can be configured or recovered.

Impact

Clearing the delegate disables all endpoint operations for this OApp. Endpoint configuration (setConfig, send and receive library selection) and inbound-message recovery (skip, clear, nilify, and burn) become uncallable, since the endpoint authorizes only the OApp itself or its delegate, and the OApp has no function to perform these calls directly. A stuck inbound message could not be cleared while the delegate is zero.

Normal send and lzReceive are unaffected, and the condition is fully recoverable: setDelegate is gated by requiresAuth and the endpoint authorizes the OApp itself for that call, so an authorized caller can restore a valid delegate in a single transaction. There is no fund risk and no bypass of any signature or authorization check; the consequence is an operational hazard that temporarily removes the contract's ability to be configured or recovered.

Recommendations

Reject the zero address in `setDelegate`, mirroring the constructor's existing `InvalidDelegate` guard, so that the nonzero delegate established at construction cannot be inadvertently cleared.

This is a consistency and defense-in-depth measure that guards only the zero sentinel; it does not, and cannot, validate that the delegate is the intended account. The delegate is an arbitrary trusted address (typically an EOA or multi-sig), so setting it to an incorrect but nonzero address carries the same class of risk and is not checkable on chain. Ensuring the delegate is the correct account remains an operational responsibility; we recommend setting it to the same trusted multi-sig as owner.

Remediation

This issue has been acknowledged by Paxos Labs.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Subdust orders are accepted instead of reverting

The documentation states that subdust orders should revert. However, when `amountDue` is rounded down to zero by `_toNormalizedDecimals`, the order is still pushed to the pending set rather than reverting. We asked whether this behavior is intentional.

Paxos Labs noted the following:

This is intentional, just because we used to revert there. But we realized this would cause messages to fail and maybe create confusion or blocking in the messaging. We figured it is better to just accept a zero amount (which ideally will not happen due to backend minimum checks before the quote signer signs) and then simply not execute those orders.

4.2. Fee-on-transfer tokens are not supported by the order accounting

The station accounts for token movements using the amounts encoded in the signed quote rather than the balances actually received. On submit, the order's `amountDue` is derived from the quoted net pulled to `offerReceiver`; on fill, `executePendingOrders` decrements the order by `fillAmount` and pulls that same amount from `wantAssetSource`. Neither path reconciles against the delta actually delivered, so for a fee-on-transfer token — where the recipient receives less than the amount sent — the recorded amounts overstate what actually moved.

Usable assets are constrained by route approvals controlled by privileged roles, so this is a token-compatibility assumption rather than a flaw in the contract. The Paxos Labs team is aware of this and considers it mitigated by route approvals being admin-gated. The subtle case is a *dormant fee switch*: a token (USDT-style) approved while charging no fee could later toggle a fee on, silently desynchronizing the accounting on an already approved route.

We are raising this point to ensure Paxos Labs remains aware of all of these cases when curating approved routes so that fee-on-transfer tokens — including those that merely retain the ability to enable a transfer fee — are not approved.

4.3. Reorg risk and LayerZero block confirmation settings

Paxos Labs raised the risk that a source-chain reorg could drop an order's submission after it has already been delivered and filled on the destination, resulting in a double spend borne by the protocol. The contract logic itself cannot prevent this; it is governed by the number of source-chain block confirmations the LayerZero security stack waits for before delivering a message. Paxos Labs described the attack vector and their planned mitigation as follows:

Attack Vector: Using Layerzero we are required to configure our own security stack, including a number of DVNs and required block confirmations. [...] if we do not enable enough, the message may be processed before a block is finalized on the source chain and may be re-organized by the chain effectively allowing a double spend. This has unique considerations depending on the chain.

Mitigation: The backend supports rate limits. Our risk is quantified by how much of pending orders are at the risk of being reorged at any given time. This is a function of how many confirmations were achieved on the source chain before that order is processed. We calculate this risk as a function of number of block confirmations and number of DVNs being used, and what type of chain the source chain is. [...] in the Executor bot, we enforce this rate limit offchain to not exceed this "expected capital at risk of reorg" at any time.

Paxos Labs then asked for a method to determine an optimized number of block confirmations that balances security and speed:

Regarding block reorgs and LZ block confirmations. We must determine an exact number, we could choose something like 10 that "sounds about right" but it would be better to have a method and explanation for picking a particular number. Do you know how other teams quantify risk here and strike a particular balance?

Recommended confirmation depth

LayerZero Labs recommends setting block confirmations for each chain "to a level where block reorganization is nearly impossible" (ref. 1). Their production DVN configuration documentation additionally provides concrete, chain-specific confirmation-depth recommendations (ref. 2).

We recommend Paxos Labs follow LayerZero's official confirmation-depth guidance as the baseline, selecting a value within the minimum-to-maximum range LayerZero recommends for each chain. This choice is a trade-off: a value toward the maximum lowers the reorg probability but increases latency, while a value toward the minimum is faster but leaves more capital exposed to a reorg-induced double spend. Because the impact of an under-confirmed message is a direct double spend, the maximum is the safest option, so we recommend choosing toward the higher end of the range to keep this risk negligible. The appropriate value is chain-specific, since reorg behavior is a property of each chain's consensus and finality model, so a single global number is not appropriate. Confirmation values should also be pinned explicitly on both the send and receive

sides rather than relying on LayerZero's defaults, which can shift and lead to asymmetric configurations or stalled messages.

The number of DVNs does not affect reorg risk

In their mitigation description, Paxos Labs notes:

We calculate this risk as a function of number of block confirmations and number of DVNs being used, and what type of chain the source chain is.

It is worth noting that the number of DVNs does not affect reorg risk. DVNs address message verification integrity and liveness — the X-of-Y-of-N model that protects against a lying or unavailable verifier — rather than source-chain finality. Reorg resistance is governed by the block confirmation count and the chain's finality, so the DVN set can be sized independently, against the threat of verifier compromise.

References

1. LayerZero Labs — [recommendation on block confirmations](#) ↗
2. LayerZero — [confirmation depth recommendations](#) ↗

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. TransitStation.sol

Function: `constructor(address _owner, Authority _authority, address _endpoint, address _protocolFeeRecipient, address _quoteSigner, address _offerReceiver, address _wantAssetSource)`

The TransitStation constructor initializes Solmate auth, the LayerZero OApp base, fee and signer configuration, asset flow endpoints, and this chain endpoint ID.

Inputs

- `_owner`
 - **Control:** Deployer-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Initial owner, Solmate auth bypass account, and initial LayerZero delegate.
- `_authority`
 - **Control:** Deployer-set.
 - **Constraints:** May be zero, though deployment passes a RolesAuthority.
 - **Impact:** Controls requiresAuth for non-owner callers.
- `_endpoint`
 - **Control:** Deployer-set.
 - **Constraints:** Must support setDelegate and eid for the deployment to complete.
 - **Impact:** Defines the LayerZero endpoint used for all messaging.
- `_protocolFeeRecipient`
 - **Control:** Deployer-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives protocol fees on order submission.
- `_quoteSigner`
 - **Control:** Deployer-set.
 - **Constraints:** Must be nonzero.

- **Impact:** Signs EIP-712 quotes accepted by submissions.
- `_offerReceiver`
 - **Control:** Deployer-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives net offer assets on submit.
- `_wantAssetSource`
 - **Control:** Deployer-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Source account from which fulfilled want assets are pulled.

Branches and code coverage

Intended branches

- Valid deployment stores all initial configuration and reads the endpoint ID.
- ☒ Test coverage

Negative behavior

- Reverts if any required address is zero, with `ZeroAddress`.
- ☐ Negative test
- Reverts if endpoint delegate setup fails or reading `eid` reverts.
- ☐ Negative test

Function call analysis

- `endpoint.setDelegate(_owner)`
 - **What is controllable?** `_endpoint` and `_owner`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value is consumed.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert propagates from the inherited `0AppAuthCore` constructor.

Function: `executePendingOrders(bytes32[] uuids, uint256[] amounts)`

The `executePendingOrders` function lets an authorized executor fill pending orders by pulling want assets from `wantAssetSource` directly to each receiver, with partial fills supported.

Inputs

- `uuids`
 - **Control:** Authorized executor-set.
 - **Constraints:** Length must equal `amounts.length`. Each UUID must exist in the pending set.
 - **Impact:** Selects pending orders to fill.
- `amounts`
 - **Control:** Authorized executor-set.
 - **Constraints:** Each amount must be less than or equal to the order amount due.
 - **Impact:** Determines how much want asset is transferred for each order.

Branches and code coverage

Intended branches

- A full fill removes the order, transfers want tokens to the receiver, and leaves no residual allowance.
 - ☒ Test coverage
- A partial fill reduces `amountDue` and keeps the order pending.
 - ☐ Test coverage
- A batch with multiple tokens checks residual allowance once per distinct want token.
 - ☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the station is paused, with `EnforcedPause`.
 - ☐ Negative test
- Reverts if the array lengths mismatch, an order is missing, a fill overfills, a token transfer fails, or residual allowance remains.
 - ☐ Negative test

Function call analysis

- `ERC20(wantAsset).safeTransferFrom(wantAssetSource, receiver, fillAmount)`

- **What is controllable?** wantAsset, wantAssetSource, receiver, and fillAmount from the selected order and executor amount.
- **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond SafeTransferLib success handling.
- **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. The state is updated before the transfer, but a revert rolls the transaction back. Reentrancy sees the order already decremented or removed during the call.
- ERC20(usedTokens[i]).allowance(wantAssetSource, address(this))
 - **What is controllable?** usedTokens[i] and wantAssetSource.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returned allowance must be zero. A token that reports a nonzero allowance causes a revert to avoid residual custody.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates after all fills, rolling the batch back.

Function: forceRemovePendingOrder(bytes32 uuid)

The forceRemovePendingOrder function lets an authorized caller remove a pending order.

Inputs

- uuid
 - **Control:** Authorized caller-set.
 - **Constraints:** Must exist in pendingOrderIds.
 - **Impact:** Selects the order to remove.

Branches and code coverage

Intended branches

- An existing pending order is removed and emitted in OrderForceRemoved.

☐ Test coverage

Negative behavior

- Reverts if requiresAuth rejects an unauthorized caller.

☐ Negative test

- Reverts if the UUID is unknown, with OrderNotFound.

☐ Negative test

Function: lzReceive(Origin _origin, bytes32 _guid, bytes _message, address _executor, bytes _extraData)

The lzReceive function is the LayerZero endpoint entry point. It authenticates the endpoint caller and configured peer before calling _lzReceive function. The _lzReceive function pushes the received order into the pending orders set.

Inputs

- `_origin`
 - **Control:** LayerZero-endpoint-supplied.
 - **Constraints:** `msg.sender` must be the configured endpoint, and `_origin.sender` must match `peers[_origin.srcEid]`.
 - **Impact:** Identifies and authenticates the remote peer path.
- `_guid`
 - **Control:** LayerZero-endpoint-supplied.
 - **Constraints:** No constraints.
 - **Impact:** Passed to the child receive implementation for event correlation.
- `_message`
 - **Control:** LayerZero-endpoint-supplied.
 - **Constraints:** Decoded by the child implementation.
 - **Impact:** Carries the application payload.
- `_executor`
 - **Control:** LayerZero-endpoint-supplied.
 - **Constraints:** No constraints.
 - **Impact:** Passed through to the child implementation.
- `_extraData`
 - **Control:** LayerZero-endpoint-supplied.
 - **Constraints:** No constraints.
 - **Impact:** Passed through to the child implementation.

Branches and code coverage

Intended branches

- An endpoint caller with a configured peer reaches `TransitStation._lzReceive` and queues the bridged order.

☐ Test coverage

Negative behavior

- Reverts if the caller is not the endpoint, with OnlyEndpoint.
 - ☐ Negative test
- Reverts if the peer is missing, with NoPeer.
 - ☐ Negative test
- Reverts if the peer does not match, with OnlyPeer.
 - ☐ Negative test
- Reverts if child `_lzReceive` reverts, leaving the message retrievable at the endpoint.
 - ☐ Negative test

Function call analysis

- `_getPeerOrRevert(_origin.srcEid)`
 - **What is controllable?** `_origin.srcEid`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
Returns the configured peer. An incorrect value makes valid messages fail or wrong peers pass if misconfigured.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert propagates as NoPeer.
- `_lzReceive(_origin, _guid, _message, _executor, _extraData)`
 - **What is controllable?** `_origin, _guid, _message, _executor, and _extraData`.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Revert or unusual control flow in the child implementation propagates to the endpoint.

Function: `pause()`

The pause function triggers the emergency stop used by `submitOrder`, `submitOrderWithPermit`, and `executePendingOrders`.

Inputs

None.

Branches and code coverage

Intended branches

- An authorized caller sets the station to paused. Calling while already paused still emits Paused because the helper `_pause` is idempotent.

☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.

☐ Negative test

Function: `quoteSend(uint32 destEID, Order order)`

The `quoteSend` function previews the LayerZero native fee for sending an encoded order to a destination endpoint ID.

Inputs

- `destEID`
 - **Control:** Arbitrary.
 - **Constraints:** Must have a configured peer, or `_quote` reverts with `NoPeer`.
 - **Impact:** Selects the destination endpoint ID for the fee quote.
- `order`
 - **Control:** Arbitrary.
 - **Constraints:** No constraints.
 - **Impact:** Provides the payload whose encoded size affects the fee.

Branches and code coverage

Intended branches

- Returns the endpoint native fee for the same payload and options used by `_sendOrder`.

☐ Test coverage

Negative behavior

- Reverts if the peer is missing, with `NoPeer`.

☐ Negative test

- Reverts if the endpoint quote fails.

☐ Negative test

Function call analysis

- `_quote(destEID, payload, options, false)`
 - **What is controllable?** `destEID`, `order`, and `messageGasLimit[destEID]`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns `MessagingFee`. Only `nativeFee` is returned. A malicious endpoint can quote an incorrect fee.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. The internal call reaches `endpoint.quote`.
- `endpoint.quote(MessagingParams(...), address(this))`
 - **What is controllable?** `destEID`, encoded `order`, `messageGasLimit[destEID]`, configured `peer`, and the endpoint contract.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned `nativeFee` is shown to callers as the amount to send with `submitOrder`. An incorrect quote can cause underpayment reverts or overpayment.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates from the view call.

Function: `recoverETH(uint256 amount)`

The `recoverETH` function sweeps stray native ETH to the owner.

Inputs

- `amount`
 - **Control:** Authorized caller-set.
 - **Constraints:** Must not exceed the contract native balance, or the low-level call fails.
 - **Impact:** Determines how much ETH is sent to owner.

Branches and code coverage

Intended branches

- An authorized caller transfers the requested ETH amount to the owner.

☐ Test coverage

Negative behavior

- Reverts if requiresAuth rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the owner call fails, with CallFailed.
 - ☐ Negative test

Function call analysis

- `owner.call{ value: amount }{""}`
 - **What is controllable?** amount and current owner.
 - **If the return value is controllable, how is it used and how can it go wrong?**
The success boolean is checked. A controllable owner contract can force a revert by rejecting ETH.
 - **What happens if it reverts, reenters or does other unusual control flow?**
Reentrancy is possible into public functions, but privileged functions still require auth and the function has no postcall state changes.

Function: recoverTokens(ERC20 token, uint256 amount)

The recoverTokens function sweeps stray ERC-20 tokens to the owner.

Inputs

- token
 - **Control:** Authorized caller-set.
 - **Constraints:** No zero-address or code check.
 - **Impact:** Selects the token contract to transfer.
- amount
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be transferable by token from this contract balance.
 - **Impact:** Determines how many tokens are sent to owner.

Branches and code coverage

Intended branches

- An authorized caller transfers the requested token amount to the owner.
 - ☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if `SafeTransferLib` reports a token transfer failure.
 - ☐ Negative test

Function call analysis

- `token.safeTransfer(owner, amount)`
 - **What is controllable?** token, owner, and amount.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. A malicious token can reenter, but there are no state changes after the call.

Function: `setAuthority(Authority newAuthority)`

The inherited `setAuthority` function replaces the Solmate authority contract used by the `requiresAuth` modifier.

Inputs

- `newAuthority`
 - **Control:** Authorized caller-set.
 - **Constraints:** No constraints.
 - **Impact:** Changes who can pass `requiresAuth` on this contract.

Branches and code coverage

Intended branches

- An authorized caller updates the authority contract.
 - ☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test

- Reverts if the current authority either reverts or denies the caller, preventing the update.
- ☐ Negative test

Function call analysis

- `authority.canCall(msg.sender, address(this), msg.sig)`
 - **What is controllable?** `msg.sender`, the current authority, and the function selector.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return value controls whether the authority can be replaced.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from the current authority propagates and can prevent authority rotation.

Function: `setDelegate(address _delegate)`

The `setDelegate` function asks the LayerZero endpoint to use a new delegate for this OApp.

Inputs

- `_delegate`
 - **Control:** Authorized caller-set.
 - **Constraints:** No constraints.
 - **Impact:** Becomes the account that can configure this OApp in the endpoint.

Branches and code coverage

Intended branches

- An authorized caller updates the endpoint delegate.
- ☒ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
- ☐ Negative test
- Reverts if the endpoint rejects the delegate update.
- ☐ Negative test

Function call analysis

- `endpoint.setDelegate(_delegate)`
 - **What is controllable?** `_delegate` and the configured endpoint.
 - **If the return value is controllable, how is it used and how can it go wrong?**
No return value is consumed.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from the endpoint reverts the call. Reentrancy is not useful because no `TransitStation` state is changed after the call in `setDelegate`.

Function: `setMessageGasLimit(uint32 eid, uint64 gasLimit)`

The `setMessageGasLimit` function configures the LayerZero executor gas used when sending orders to a destination endpoint ID.

Inputs

- `eid`
 - **Control:** Authorized caller-set.
 - **Constraints:** No range check.
 - **Impact:** Selects the destination endpoint ID.
- `gasLimit`
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Controls gas encoded into LayerZero options for destination `lzReceive`.

Branches and code coverage

Intended branches

- An authorized caller sets the gas limit for a destination endpoint ID.

☒ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.

☐ Negative test

Function: setOfferReceiver(address newOfferReceiver)

The setOfferReceiver function updates where net offer assets are sent when users submit orders.

Inputs

- newOfferReceiver
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives future net offer deposits.

Branches and code coverage

Intended branches

- An authorized caller updates offerReceiver and emits an event.
 - ☐ Test coverage

Negative behavior

- Reverts if requiresAuth rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the offer receiver is zero, with ZeroAddress.
 - ☐ Negative test

Function call analysis

- authority.canCall(msg.sender, address(this), msg.sig)
 - **What is controllable?** msg.sender, the current authority, and the function selector.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return value gates the call. A malicious or misconfigured authority can deny valid callers or allow unexpected callers.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from the authority propagates before the function body executes.

Function: setPeer(uint32 _eid, bytes32 _peer)

The setPeer function records or clears the trusted LayerZero peer for a source or destination endpoint ID.

Inputs

- `_eid`
 - **Control:** Authorized caller-set.
 - **Constraints:** No range check.
 - **Impact:** Selects the endpoint ID whose peer mapping is changed.
- `_peer`
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Defines which remote OApp sender is trusted for that endpoint ID.

Branches and code coverage

Intended branches

- An authorized caller sets or clears a peer.
 - ☒ Test coverage:
 - `test/transit/DeployTransitStation.t.sol::test_deployment` verifies the peer set during deployment

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
- ☐ Negative test

Function call analysis

- `_setPeer(_eid, _peer)`
 - **What is controllable?** `_eid` and `_peer`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
 - **What happens if it reverts, reenters or does other unusual control flow?** An internal call cannot reenter. It updates storage and emits `PeerSet`.

Function: `setProtocolFeeRecipient(address recipient)`

The `setProtocolFeeRecipient` function updates where protocol fees are sent during order submission.

Inputs

- recipient
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Receives future protocol-fee transfers.

Branches and code coverage

Intended branches

- An authorized caller updates protocolFeeRecipient and emits an event.
 - ☐ Test coverage

Negative behavior

- Reverts if requiresAuth rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the recipient is zero, with ZeroAddress.
 - ☐ Negative test

Function: setQuoteSigner(address signer)

The setQuoteSigner function rotates the backend key that signs accepted EIP-712 quotes.

Inputs

- signer
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Defines which signatures can authorize future orders.

Branches and code coverage

Intended branches

- An authorized caller updates quoteSigner, and future submissions use the new key.
 - ☒ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the signer is zero, with `ZeroAddress`.
 - ☐ Negative test

Function: `setRouteApprovals(Route[] routes, bool[] approved)`

The `setRouteApprovals` function batch-updates the global directional route allowlist used on both source submission and destination receive.

Inputs

- `routes`
 - **Control:** Authorized caller-set.
 - **Constraints:** Length must equal `approved.length`.
 - **Impact:** Selects destination EID, offer asset, and want-asset tuples to update.
- `approved`
 - **Control:** Authorized caller-set.
 - **Constraints:** Length must equal `routes.length`.
 - **Impact:** Sets each route as enabled or disabled.

Branches and code coverage

Intended branches

- An authorized caller batch-enables or disables routes.
 - ☒ Test coverage:
`test/transit/DeployTransitStation.t.sol::_ownerSetup` enables a route, and submit tests depend on it

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the array lengths mismatch, with `LengthMismatch`.
 - ☐ Negative test

Function call analysis

- `authority.canCall(msg.sender, address(this), msg.sig)`
 - **What is controllable?** `msg.sender`, the current `authority`, and the function selector.
 - **If the return value is controllable, how is it used and how can it go wrong?** The boolean return value gates the call. A malicious or misconfigured `authority` can deny valid callers or allow unexpected callers.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from the `authority` propagates before the function body executes.

Function: `setWantAssetSource(address newWantAssetSource)`

The `setWantAssetSource` function updates the account from which the station pulls want assets during execution.

Inputs

- `newWantAssetSource`
 - **Control:** Authorized caller-set.
 - **Constraints:** Must be nonzero.
 - **Impact:** Future fills pull want assets from this address.

Branches and code coverage

Intended branches

- An authorized caller updates `wantAssetSource` and emits an event.
 - ☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
 - ☐ Negative test
- Reverts if the want-asset source is zero, with `ZeroAddress`.
 - ☐ Negative test

Function: `submitOrder(Quote quote, bytes signature)`

The `submitOrder` function accepts a backend-signed quote, collects the offer asset, and either queues the order locally or bridges it to the destination station.

Inputs

- `quote`
 - **Control:** Arbitrary submitter-provided, constrained by `quoteSigner` signature and route approval.
 - **Constraints:** Deadline must be live, route must be approved, receiver must be nonzero, amount due must be nonzero, protocol fee must be within cap, and total fees must be lower than offer amount.
 - **Impact:** Defines the asset pair, amounts, receiver, fees, destination, and replay salt for the order.
- `signature`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Must recover exactly `quoteSigner` for the EIP-712 digest.
 - **Impact:** Authorizes the quoted order terms.
- `msg.value`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** For cross-chain orders, must exactly match the native LayerZero fee expected by `_payNative`.
 - **Impact:** Funds the LayerZero send path when `quote.route.destEID != thisChainEID`.

Branches and code coverage

Intended branches

- A valid same-chain quote pulls the offer asset and queues a pending order.
 - ☒ Test coverage: `test/transit/DeployTransitStation.t.sol::test_singleChainSubmitAndExecute` covers local queueing
- A valid cross-chain quote pulls the offer asset and dispatches a LayerZero message.
 - ☒ Test coverage: `test/transit/DeployTransitStation.t.sol::test_crossChainSubmitAttemptsBridge` covers bridge dispatch

Negative behavior

- Reverts if `requiresAuth` rejects the caller because the function is not public or the caller is not allowed.
 - ☐ Negative test
- Reverts if the station is paused, with `EnforcedPause`.
 - ☐ Negative test
- Reverts if the quote is expired, route is unapproved, receiver is zero, amount due is zero,

protocol fee is too high, fees exceed the offer, signer is invalid, or digest is reused.

- ☐ Negative test
- Reverts if a cross-chain submission has no gas limit or the native fee is incorrect.
- ☐ Negative test
- Reverts if the offer-token transfer fails.
- ☐ Negative test

Function call analysis

- `_submitOrder(quote, signature)`
 - **What is controllable?** `quote`, `signature`, and `msg.value`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns `uuid`, the EIP-712 digest. If digest construction or validation is incorrect, orders can be replayed or rejected incorrectly.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. The internal flow performs external token and endpoint calls before returning.
- `offer.safeTransferFrom(msg.sender, protocolFeeRecipient, quote.protocolFee)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote.protocolFee`, and `current.protocolFeeRecipient`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back `usedDigests[digest]`. A malicious token can reenter, but the digest is already marked used during the reentrant frame.
- `offer.safeTransferFrom(msg.sender, quote.integratorFeeReceiver, quote.integratorFee)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote.integratorFeeReceiver`, and `quote.integratorFee`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the full submission.
- `offer.safeTransferFrom(msg.sender, offerReceiver, net)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote-fee` values used to compute `net`, and the current `offerReceiver`.
 - **If the return value is controllable, how is it used and how can it go wrong?**

No return value is consumed beyond SafeTransferLib success handling.

- **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the full submission.
- `endpoint.send{ value: messageValue }(...)`
 - **What is controllable?** `quote.route.destEID`, encoded order, `messageGasLimit[destEID]`, `msg.value`, configured peer, and `msg.sender` as the refund address.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned receipt guid is emitted in `OrderBridged`. A malicious endpoint can return misleading receipt data.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates. The order is not queued locally before this call, so rollback restores token transfer and digest state.

Function: `submitOrderWithPermit(Quote quote, bytes signature, uint256 permitDeadline, uint8 v, bytes32 r, bytes32 s)`

The `submitOrderWithPermit` function first tries an EIP-2612 permit for the offer token, then submits the signed quote through the same core path as `submitOrder`.

Inputs

- `quote`
 - **Control:** Arbitrary submitter-provided, constrained by `quoteSigner` signature and route approval.
 - **Constraints:** Same constraints as `submitOrder`.
 - **Impact:** Defines the order terms and offer token to permit.
- `signature`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Must recover exactly `quoteSigner` for the quote digest.
 - **Impact:** Authorizes the quoted order terms.
- `permitDeadline`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Checked by the offer-token permit implementation.
 - **Impact:** Sets the permit expiry.
- `v`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Checked by the offer-token permit implementation.
 - **Impact:** Permit signature component.

- `r`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Checked by the offer-token permit implementation.
 - **Impact:** Permit signature component.
- `s`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** Checked by the offer-token permit implementation.
 - **Impact:** Permit signature component.
- `msg.value`
 - **Control:** Arbitrary submitter-provided.
 - **Constraints:** For cross-chain orders, must exactly match the native LayerZero fee expected by `_payNative`.
 - **Impact:** Funds the LayerZero send path.

Branches and code coverage

Intended branches

- Valid permit plus quote submits an order.
 - ☐ Test coverage
- Failed permit falls back to an allowance check and can still submit if the allowance is sufficient.
 - ☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects the caller because the function is not public or the caller is not allowed.
 - ☐ Negative test
- Reverts if the station is paused, with `EnforcedPause`.
 - ☐ Negative test
- Reverts if the permit fails and allowance is insufficient, with `PermitFailedAndAllowanceTooLow`.
 - ☐ Negative test
- Reverts if any `submitOrder` validation or transfer failure occurs.
 - ☐ Negative test

Function call analysis

- `ERC20(quote.route.offerAsset).permit(msg.sender, address(this), quote.offerAmount, permitDeadline, v, r, s)`
 - **What is controllable?** `quote.route.offerAsset`, `msg.sender`, `quote.offerAmount`, `permitDeadline`, `v`, `r`, and `s`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed. A nonstandard token can revert or behave unexpectedly.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert is caught and triggers the allowance fallback.
- `offer.allowance(msg.sender, address(this))`
 - **What is controllable?** `quote.route.offerAsset` and `msg.sender`.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned allowance decides whether a failed permit is acceptable. A misleading token can cause false success or false revert.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert from allowance propagates from the catch block.
- `_submitOrder(quote, signature)`
 - **What is controllable?** `quote`, `signature`, and `msg.value`.
 - **If the return value is controllable, how is it used and how can it go wrong?** Returns `uuid`, the EIP-712 digest.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates after any successful permit state change.
- `offer.safeTransferFrom(msg.sender, protocolFeeRecipient, quote.protocolFee)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote.protocolFee`, and `current.protocolFeeRecipient`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the quote submission. The prior permit approval state is also reverted because it is in the same transaction.
- `offer.safeTransferFrom(msg.sender, quote.integratorFeeReceiver, quote.integratorFee)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote.integratorFeeReceiver`, and `quote.integratorFee`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the full transaction.

- `offer.safeTransferFrom(msg.sender, offerReceiver, net)`
 - **What is controllable?** `msg.sender`, `quote.route.offerAsset`, `quote-fee` values used to compute `net`, and current `offerReceiver`.
 - **If the return value is controllable, how is it used and how can it go wrong?** No return value is consumed beyond `SafeTransferLib` success handling.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the full transaction.
- `endpoint.send{ value: messageValue }(...)`
 - **What is controllable?** `quote.route.destEID`, `encoded order`, `messageGasLimit[destEID]`, `msg.value`, `configured peer`, and `msg.sender` as refund address.
 - **If the return value is controllable, how is it used and how can it go wrong?** The returned receipt `guid` is emitted in `OrderBridged`. A malicious endpoint can return misleading receipt data.
 - **What happens if it reverts, reenters or does other unusual control flow?** A revert propagates and rolls back the permit plus submission transaction.

Function: `transferOwnership(address newOwner)`

The inherited `transferOwnership` function changes the owner address that bypasses authority checks.

Inputs

- `newOwner`
 - **Control:** Authorized caller-set.
 - **Constraints:** No constraints.
 - **Impact:** Receives owner privileges and can pass `requiresAuth` as owner.

Branches and code coverage

Intended branches

- An authorized caller transfers ownership.
- ☒ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.
- ☐ Negative test
- Reverts if the caller lacks authorization before a zero-address ownership transfer. The

inherited ownership logic otherwise accepts address (0).

☐ Negative test

Function: `unpause ( )`

The unpause function resumes submissions and execution after an emergency stop.

Inputs

None.

Branches and code coverage

Intended branches

- An authorized caller clears the pause flag when the station is paused.

☐ Test coverage

Negative behavior

- Reverts if `requiresAuth` rejects an unauthorized caller.

☐ Negative test

- Reverts if called while not paused, with `ExpectedPause`.

☐ Negative test

6. Assessment Results

During our assessment on the scoped Nucleus Boring Vault contracts, we discovered one finding, which was informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.